

Programmieren 1



03: Verzweigungen

Sommersemester 2025

Simon Dewert

HAW Hamburg

Fakultät Design, Medien und Information

E52, Finkenau 35, 22081 Hamburg

simon.dewert@haw-hamburg.de

Hausaufgabe: Eure Lösungen

```
String nN,gender,geburtsdatum,beruf;
Scanner meinScanner = new Scanner(System.in);
StringBuilder sb = new StringBuilder();

System.out.print("Name:\n");
String str = meinScanner.next();
sb.append(str+" ");
System.out.print("Nachname:\n");
nN= meinScanner.next();
sb.append(nN+" ");
System.out.print("Geschlecht:\n");
gender= meinScanner.next();
sb.append(gender+" ");
System.out.print("Geburtsdatum:\n");
geburtsdatum= meinScanner.next();
sb.append("geb. "+geburtsdatum+" ");
System.out.print("Beruf:\n");
beruf= meinScanner.next();
sb.append(beruf);
```

```
// Variablen Deklaration
String eingabe, ausgabe;
StringBuilder sb = new StringBuilder();

Scanner scan = new Scanner(System.in);
System.out.print("Name: \t\t");
eingabe = scan.nextLine();
sb.append(eingabe + " ");

System.out.print("Nachname: \t");
eingabe = scan.nextLine();
sb.append(eingabe + " ");

System.out.print("Geschlecht: \t");
eingabe = scan.nextLine();
sb.append(eingabe + " ");

System.out.print("Geburtsdatum: \t");
eingabe = scan.nextLine();
sb.append(eingabe + " ");

System.out.print("Beruf: \t\t");
eingabe = scan.nextLine();
sb.append(eingabe);
scan.close();
```

```
sb.append(name + " ");
sb.append(nachname + "; ");
sb.append(geschlecht + "; ");
sb.append("geb. " + geburtsdatum + "; ");
sb.append(beruf);
```

```
StringBuilder str = new StringBuilder();
Scanner meinScanner = new Scanner(System.in);
System.out.println("Name ");
str.append(meinScanner.next()+ " ");
System.out.println("Nachname ");
str.append(meinScanner.next()+ " ");
System.out.println("Geschlecht ");
str.append(meinScanner.next()+ " ");
System.out.println("Geburtsdatum ");
str.append(meinScanner.next()+ " ");
System.out.println("Beruf ");
str.append(meinScanner.next()+ " ");
```

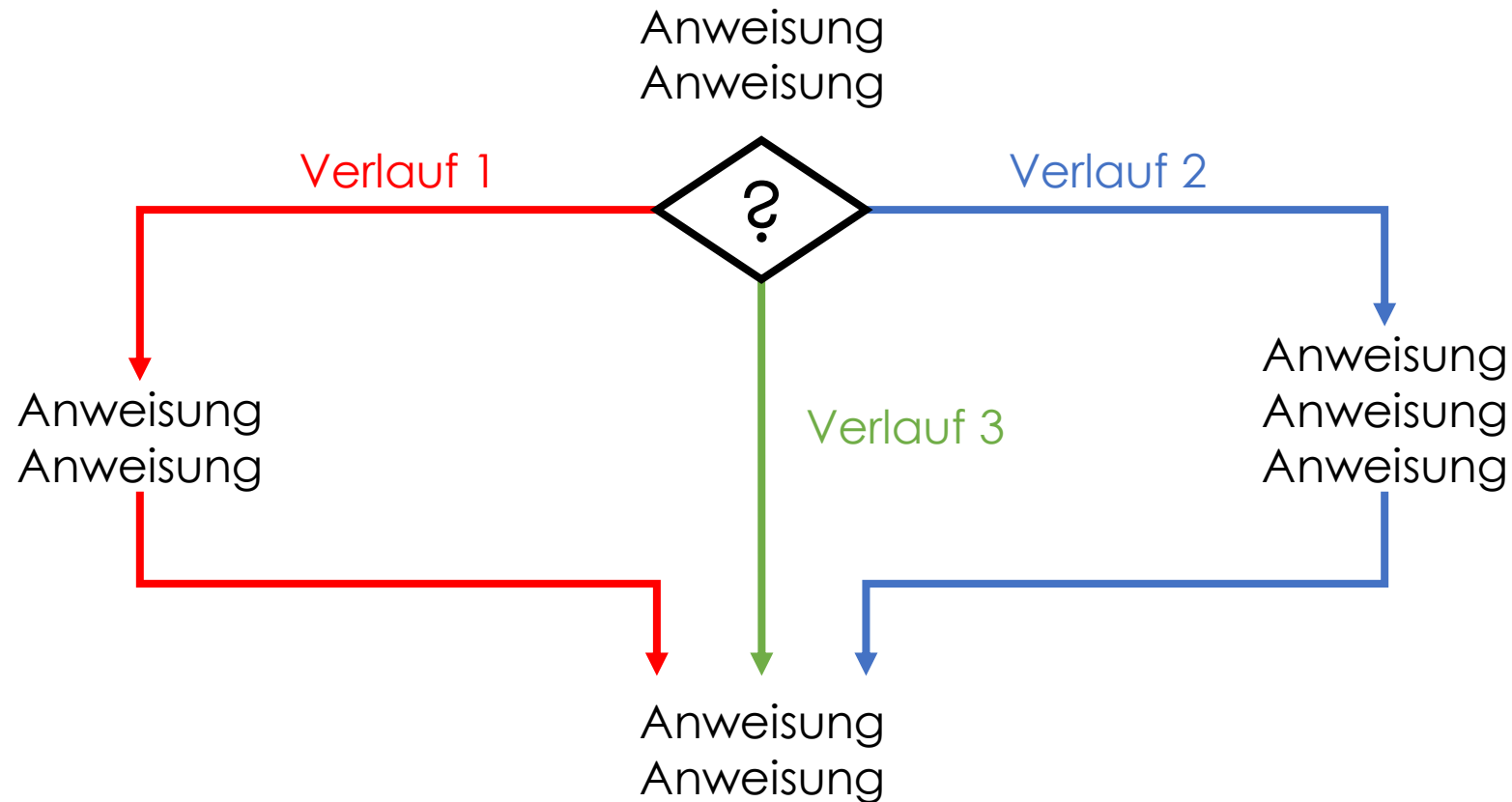
```
StringBuilder sb = new StringBuilder(name);
sb.append(" ").append(nachname).append("; ").append(geschlecht).append("; geb. ").append(geburtsdatum)
    .append("; ").append(beruf);
```

Übersicht dieser Lehreinheit

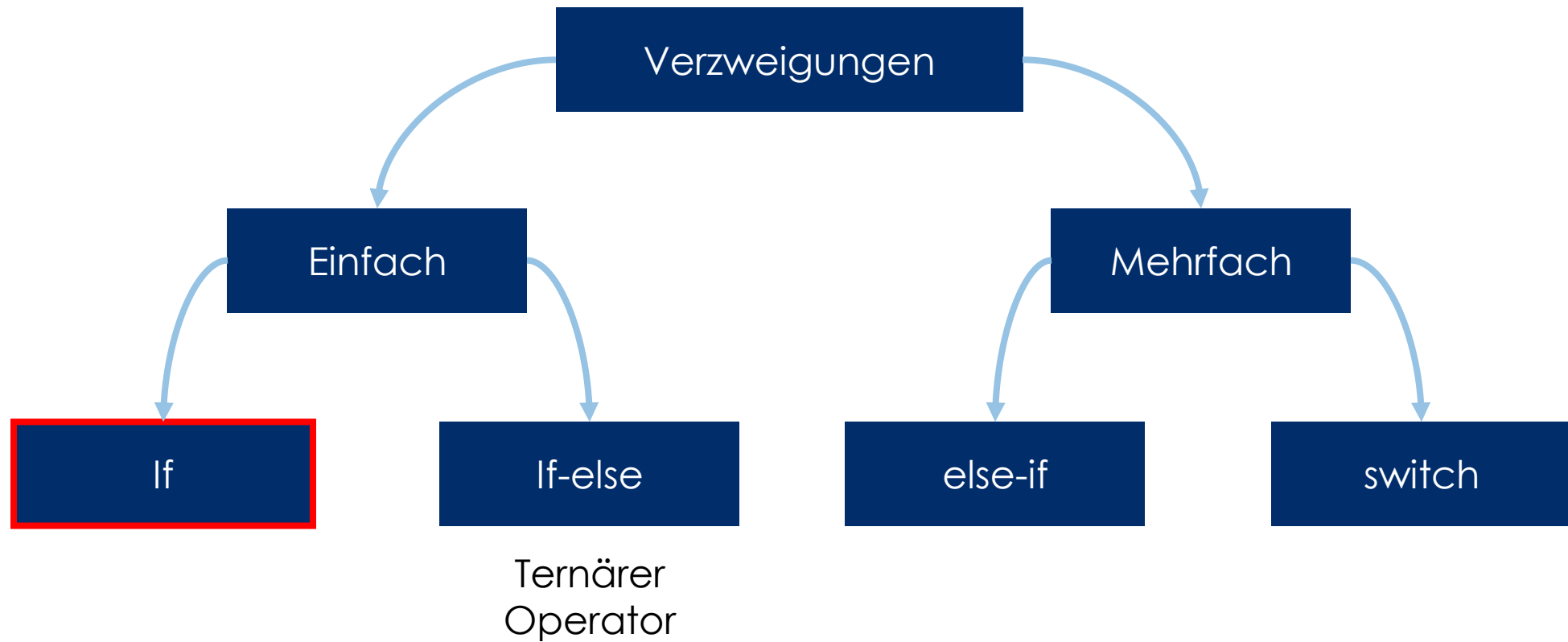
1. Funktion von Verzweigungen
2. Verzweigungs-Topologie
3. If-Verzweigung
4. If-else-Verzweigung
5. Ternäre Verzweigung
6. Else-if-Verzweigung
7. Switch-Verzweigung

Funktion von Verzweigungen

Verzweigungen dienen der **Kontrolle des Programmflusses**:



Verzweigungs-Topologie



If-Verzweigung

Mit der **If**-Verzweigung kann die Erfüllung einer einfachen **Bedingung** in Form eines **booleschen Ausdrucks** abgeprüft werden.

Ist die Bedingung erfüllt, wird ein mit geschweiften Klammern begrenzter Anweisungsblock ausgeführt. Ansonsten nicht:

```
if (Ausdruck)
{
    Anweisungen;
}
```

Boolesche Variable, relationaler
Ausdruck oder logischer Ausdruck
(true oder false)

* kann wiederum Verzweigungen enthalten

Aufgabe 3.1



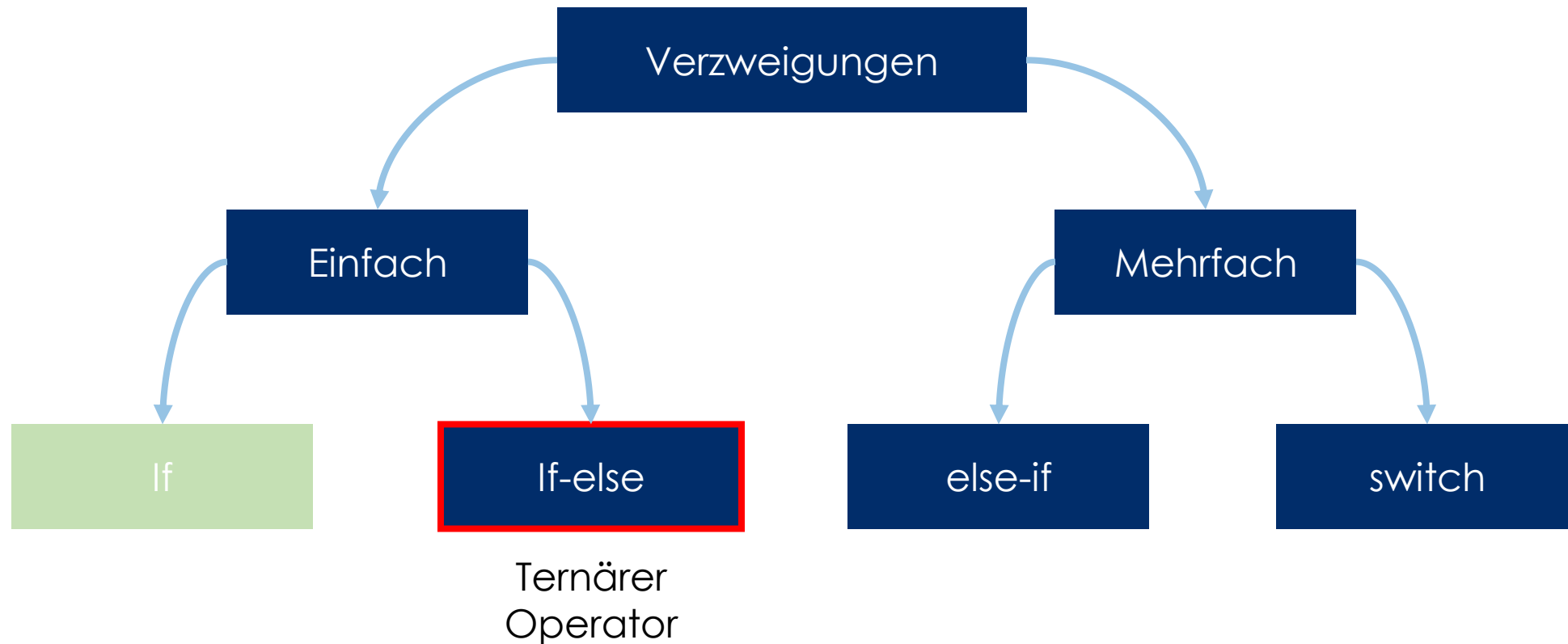
Legen Sie zunächst ein neues Java-Projekt namens **vorlesung_03** an!
Schreiben Sie dann ein Programm **AbschreibeTest**, dass den Benutzer dazu auffordert einen vorgegebenen Text abzutippen.

Den abzutippenden Text dürfen Sie frei wählen.
Nach der Eingabe soll geprüft werden, ob richtig abgeschrieben wurde. **Nur bei einem Fehler** soll Feedback gegeben werden.

Beispiel:

Bitte Tippen Sie - Keep It Simple sTupID:	keep it simpel stupid	Eingabe
Die Eingabe war leider nicht korrekt!		Ausgabe

Verzweigungs-Topologie



If-else-Verzweigung

Mit der if-else-Verzweigung kann ebenfalls die Erfüllung einer einfachen **Bedingung** in Form eines boolschen Ausdrucks abgeprüft werden.

Zusätzlich zur If-Verzweigung bietet die if-else-Verzweigung einen **alternativen Anweisungsblock**, der nur ausgeführt wird, wenn der Ausdruck **nicht erfüllt** ist.

```
if (Ausdruck) {  
    Anweisungen;           // Bedingter Anweisungsblock*  
} else {  
    Anweisungen;           // Alternativer Anweisungsblock*  
}
```

*kann wiederum Verzweigungen enthalten

Aufgabe 3.2



Passen Sie ihr Programm **IntervallChecker** so **an**, dass es differenziert Rückmeldung gibt, ob die Zahl, die eingegeben wurde, **unterhalb**, **innerhalb** oder **oberhalb** des Intervalls **[10,20]** liegt!

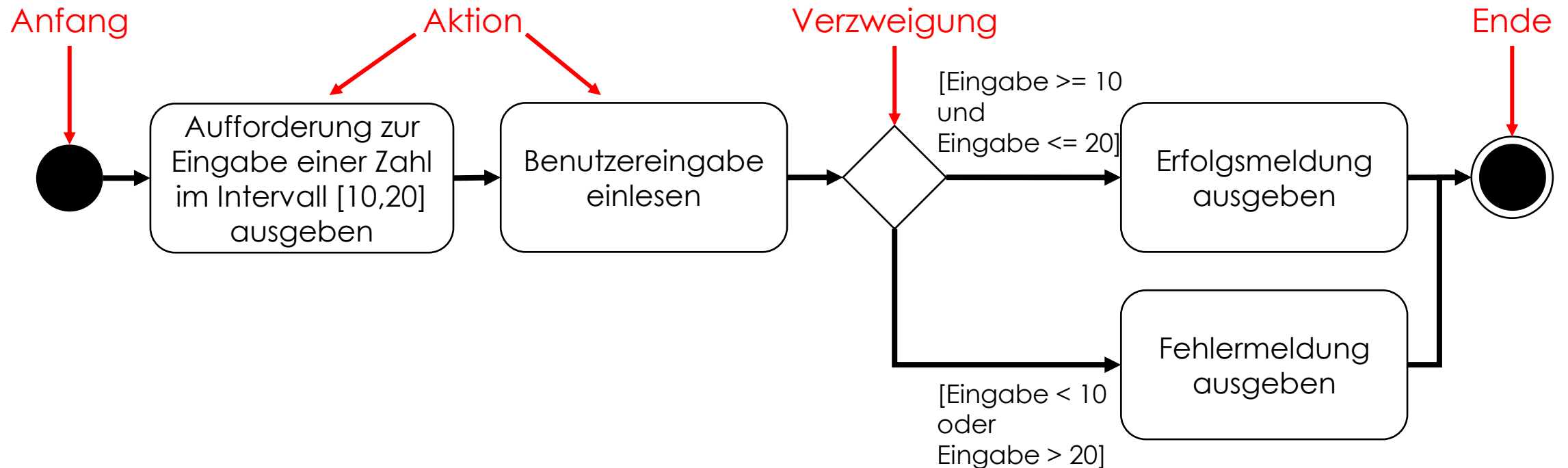
Beispiel:

Geben Sie eine Ganzzahl innerhalb des Intervalls [10,20] ein: 22 Eingabe
22 liegt OBERHALB des Intervalls [10,20]. Ausgabe

Angabe, ob **UNTERHALB**, **INNERHALB** oder **OBERHALB** des Intervalls

UML 2.0 Aktivitätsdiagramm

Aktivität „Intervall checken“ anpassen



Aufgabe 3.3



Schreiben Sie dann ein Programm **Begrueßung**, das den Benutzer zur Eingabe des **Vornamens**, des **Nachnamens** und des **Geschlechts** (m/w) auffordert. Abschließend soll das Programm fragen, ob es den Benutzer **duzen** darf.

In Abhängigkeit von den Gemachten Eingaben soll das Programm dann den Benutzer korrekt begrüßen:

```
Hallo <Vorname>, ich heiße Dich willkommen!
```

Ausgabe

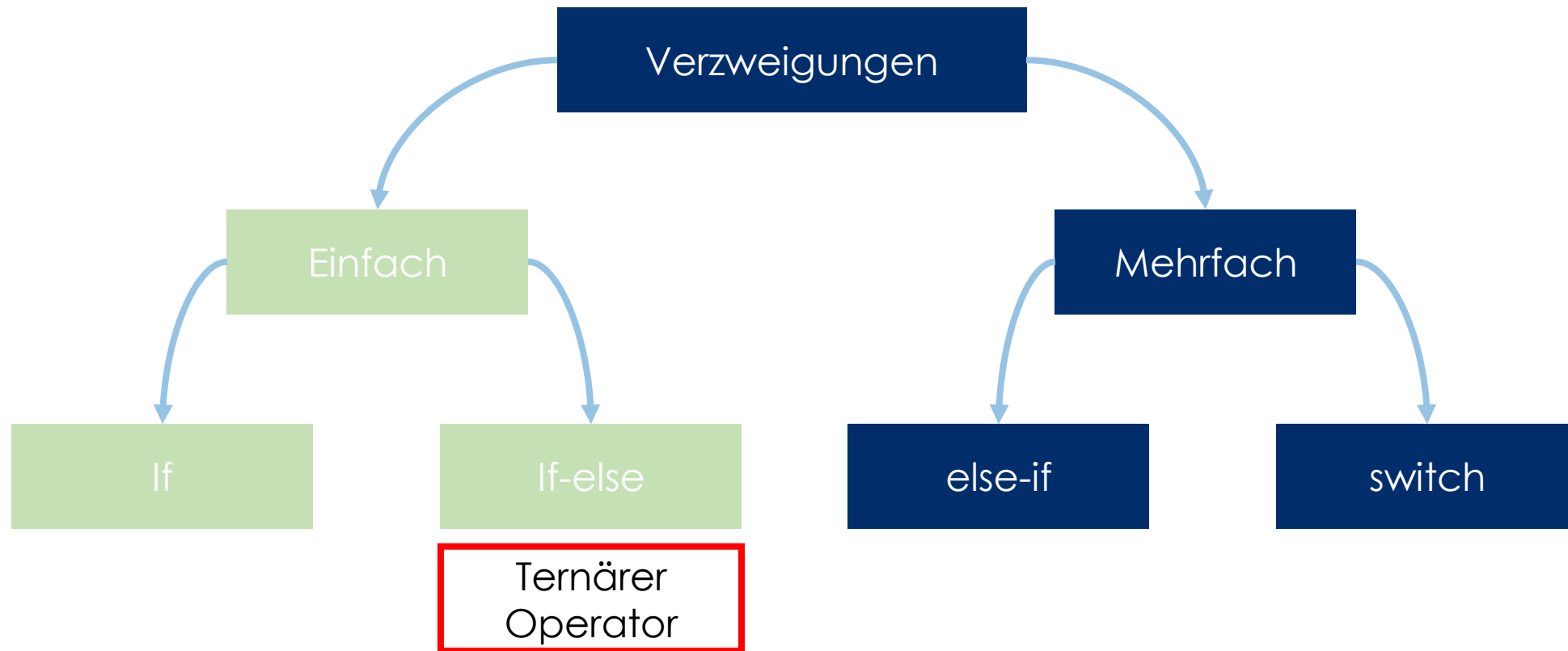
```
Guten Tag Frau <Nachname>, ich heiße Sie willkommen!
```

Ausgabe

```
Guten Tag Herr <Nachname>, ich heiße Sie willkommen!
```

Ausgabe

Verzweigungs-Topologie



Ternärer Operator

In Kombination mit Zuweisungsoperatoren kann alternativ zur if-else-Verzweigung auch der **kompaktere** ternäre Operator **?** : verwendet werden:

```
boolean kriterium = true;
```

```
int zahl = 0;
```

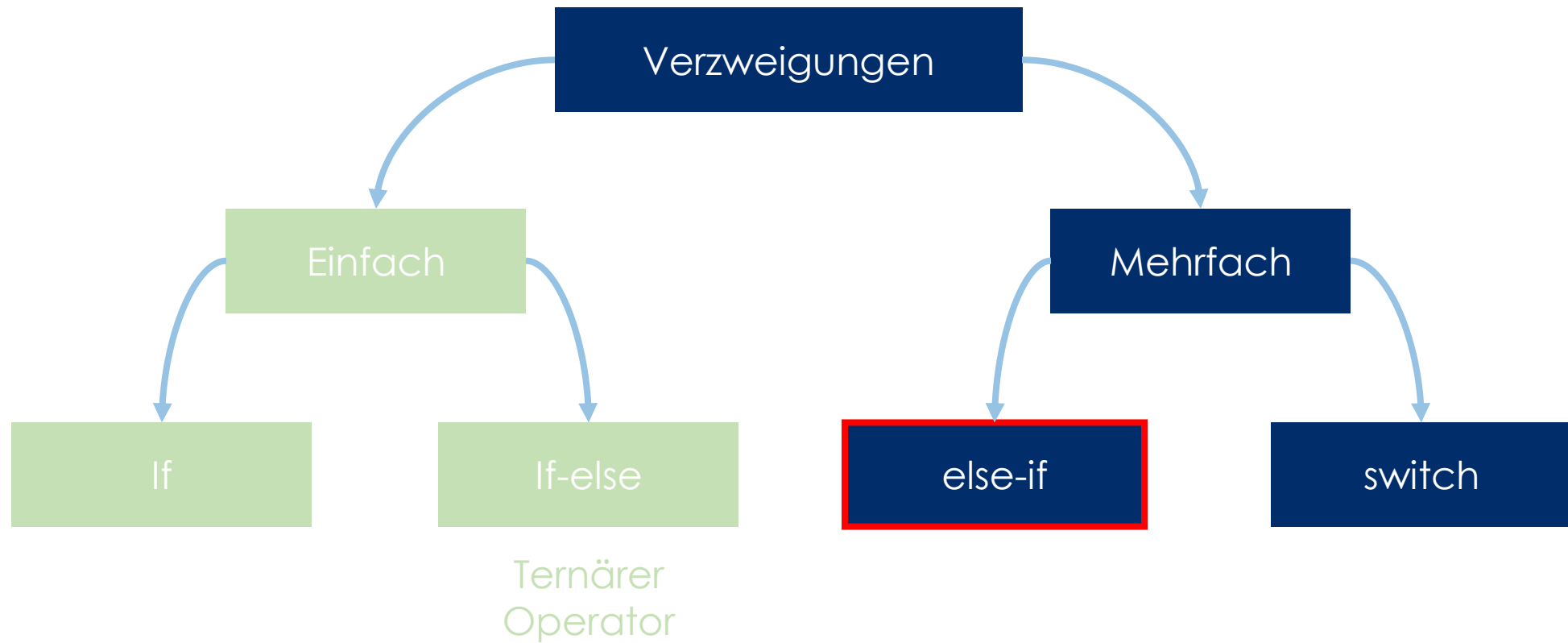
```
zahl = kriterium ? 1 : 2;
```



```
if (kriterium) {  
    zahl = 1;  
} else {  
    zahl = 2;  
}
```

Aus Gründen der Lesbarkeit wird die if-else-Verzweigung bevorzugt!

Verzweigungs-Topologie



Else-if-Verzweigung

Die else-if-Verzweigung entspricht einer if-else Verzweigung, der eine oder mehrere **weitere if-else-Verzweigungen angeschlossen** werden, um ein **sequentielles** Abprüfen **mehrerer Bedingungen** zu erlauben.

```
if (Ausdruck1) {  
    Anweisungen1;  
}  
else if (Ausdruck2) {  
    Anweisungen2;  
}  
else if (Ausdruck3) {  
    Anweisungen3;  
[...]  
}  
else {  
    AnweisungenX;  
}
```

Hinweis

Sobald ein Ausdruck wahr ist, wird der dazugehörige Anweisungsblock ausgeführt und die **restlichen** Abfragen werden **übersprungen**.

Aufgabe 3.4



Schreiben Sie dann ein Programm **Zahlenschreiber**, das dazu auffordert, eine beliebige ganze Zahl zwischen 1 und 5 einzugeben. Lassen Sie das Programm die eingegebene Zahl dann als Wort ausgeben!

Wird etwas anderes als eine Zahl zwischen 1 und 5 eingegeben, soll „Zahl unbekannt“ ausgegeben werden.

Geben Sie eine Ganzzahl zwischen 1 und 5 ein:

4

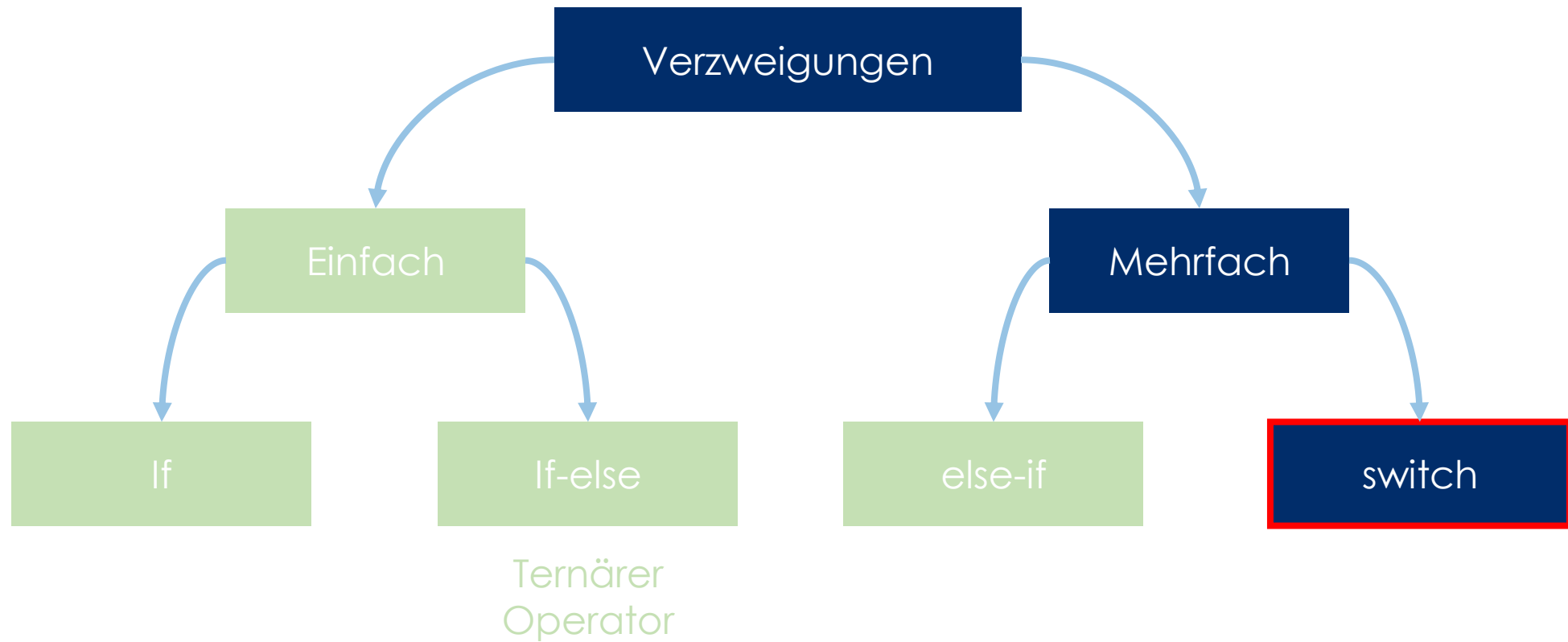
Eingabe

Vier

Ausgabe

eingeegebene Zahl als Wort

Verzweigungs-Topologie



Switch-Verzweigung

Die switch-Verzweigung stellt eine **Spezialisierung der else-if-Verzweigung** dar. Mit ihr kann eine numerische Variable, ein numerischer Ausdruck oder ein String **mit einer Reihe von Werten passenden Typs** verglichen werden:

```
switch (Ausdruck) {  
    case WERT_1: CASE  
        Anweisungen;  
        break;  
    case WERT_2: CASE  
        Anweisungen;  
        break;  
    [...]  
    default:  
        Anweisungen;  
}
```

Hinweis

Betrachtung unterschiedlicher „Fälle“ (engl. „cases“) **break**; verlässt die gesamte switch-Verzweigung
Ohne **break**, werden die nachfolgenden Anweisungen der noch folgenden Fälle bis zur nächsten **break**-Anweisung auch noch ausgeführt!

default: ist der Standard-Fall
(wenn KEINER der vorherigen Fälle zugetroffen hat)

Switch-Verzweigung

Es können auch mehrere Werte mit Komma zusammengefasst werden.

```
switch (Ausdruck) {  
    case 1,2,3,4,5,6,7,8:  
        Anweisungen;  
        break;  
    case 9,10:  
        Anweisungen;  
        break;  
    [...]  
    default:  
        Anweisungen;  
}
```

Aufgabe 3.5



Schreiben Sie dann ein Programm **NotenChecker**, das dazu auffordert, eine Note zwischen 1 und 5 einzugeben. Lassen Sie das Programm dann aussagen, **ob die Note zum Bestehen reicht!**

Hinweis: Nur Noten 1 bis 4 bestehen.

Verwenden Sie die **switch-Verzweigung** und codieren Sie möglichst effizient!

Geben Sie ihre Note ein:

2

Eingabe

Sie haben bestanden!

Ausgabe

Viele Dank für Ihre Aufmerksamkeit



Hausaufgabe

Erweitern Sie das Programm **NotenChecker**, sodass bei invaliden Eingaben mit „ungültige Note“ quittiert wird.

Außerdem sollen die Noten entsprechend ihrer Textform ausgegeben werden (1 = mit sehr gut bestanden, 2 = mit gut bestanden, usw. ...).

Geben Sie ihre Note ein: 3

Eingabe

Sie haben mit der Note befriedigend bestanden!

Ausgabe